



Department of ICT

A major project final defense report on:

SyncWrite: A collaborative document writing platform

Project Members:

Aayush Karki 200101
Anish Bista 200105
Pragya Pokharel 200123
Rabin Mishra 200128
Pankaj Kumar Yadav 200121

Submitted to:

Department of ICT
Cosmos College of
Management and
Technology

Date : 2082/04/07



Project Overview

- Real-time collaborative document writing platform for simultaneous co-authoring of rich text
- Built using modern development framework & tools
- Technologies: Flutter frontend, Express js backend, Mongodb
- Features: Real-time updates, Note sharing, Text formatting, styling & many more



Problem & Motivation



- Inefficient Collaborative Workflows (version conflicts, delays).
- Lack of Real-time Synchronization in many editors.
- Platform Dependency limiting accessibility
- Complex Secure Authentication without reliance on Firebase.



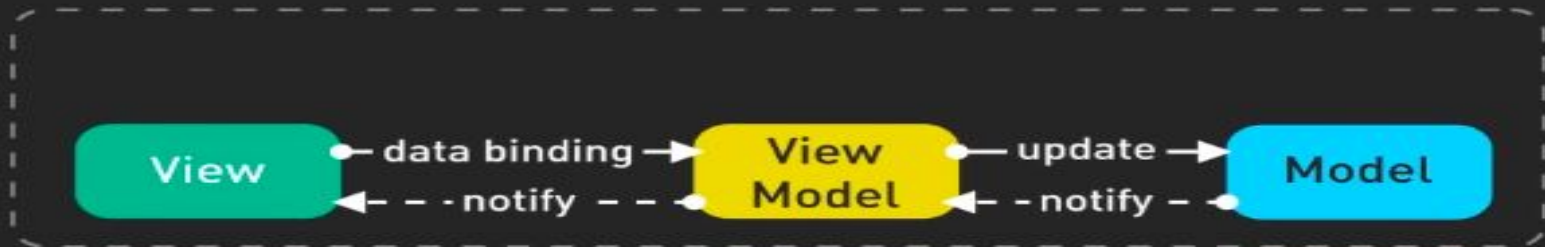
Key Achievements

- Developed cross-platform (Web, Android, iOS) editor (Flutter frontend, Node.js backend).
- Implemented secure Google Sign-In with Google Cloud OAuth (no Firebase).
- Integrated rich text editor (Flutter Quill) with formatting.
- Enabled real-time collaboration via Socket.IO.
- Implemented efficient auto-save and persistent user sessions (JWT).



Client Architecture

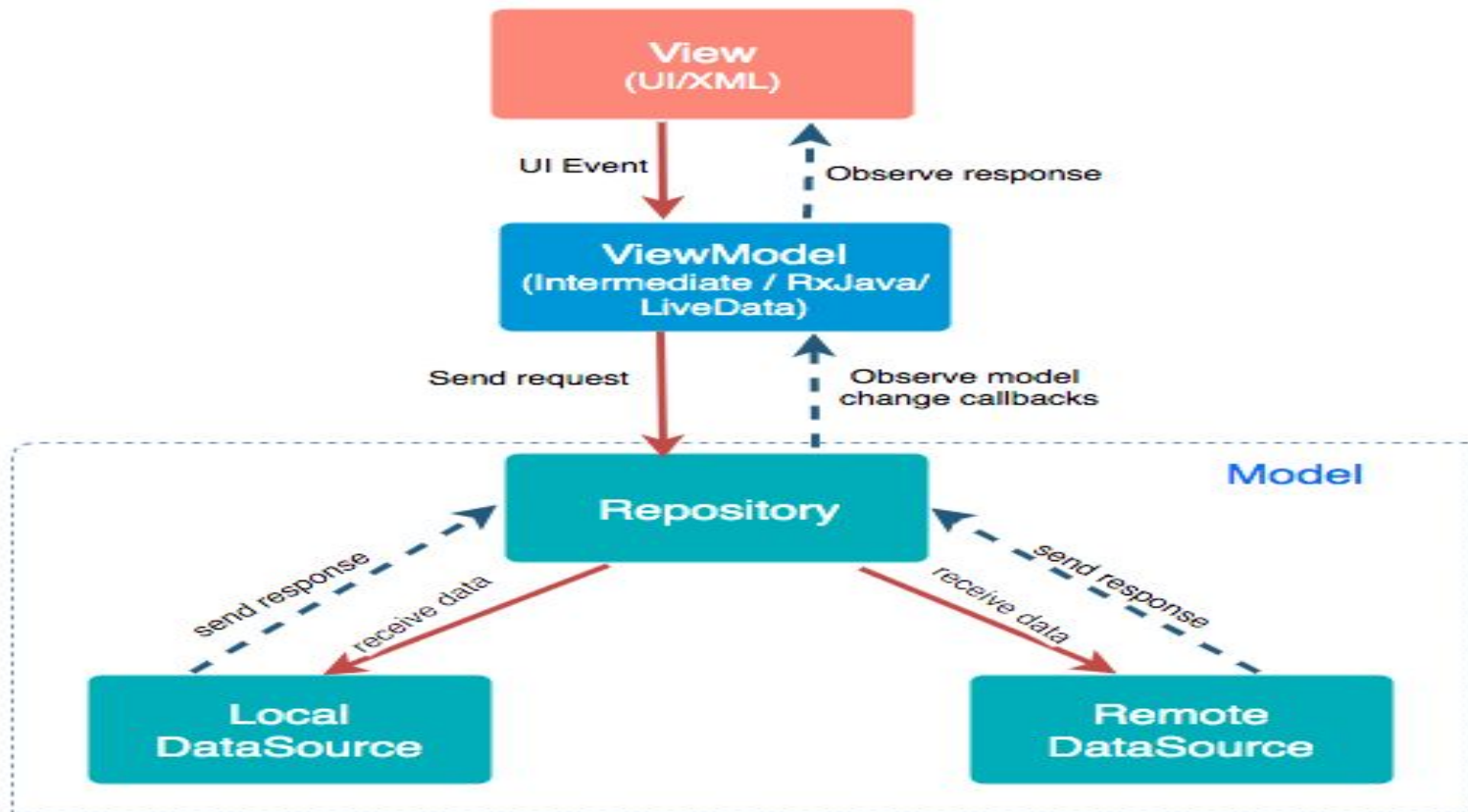
MVVM: Model View View-Model

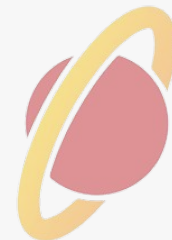


- Reduces explicit update logic
- View Model handles syncing raw model data to reflect UI changes automatically
- By handling bidirectional syncing between view and model data binding streamlines updates that would otherwise required boilerplate code .
- ViewModel -> POCO(Plain Old CLR Object) to make it unit testable with loose coupling approach.

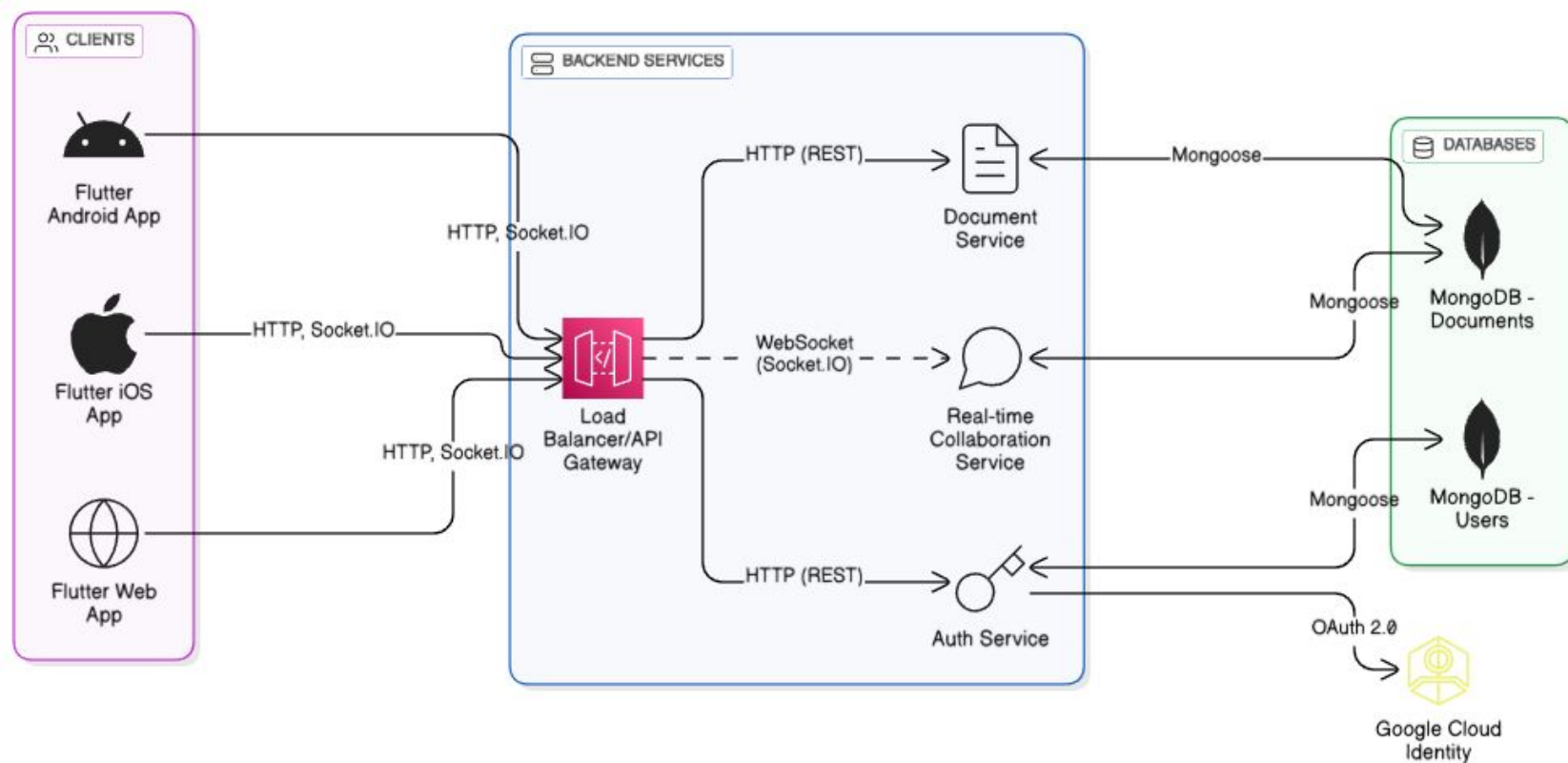


How It Actually Works





Working of App





Working of App

1. User Onboarding & Authentication:

- Users access SyncWrite (Web, Android, iOS Flutter app).
- They sign in securely using their existing Google accounts via Google Sign-In.
- The Flutter app uses the `google_sign_in` package, and the backend verifies the Google ID token with Google Cloud Identity.
- Upon successful verification, the Node.js backend issues a JSON Web Token (JWT).
- This JWT is stored locally on the client to maintain persistent user sessions, allowing users to stay logged in across app restarts.



Working of App

2. Document Management (Dashboard):

- After logging in, users land on their document dashboard.
- Here, they can view a list of all documents they've created or that have been shared with them.
- Users can create new documents or open existing ones.



Working of App



3. Collaborative Document Editing:

- When a user opens a document, they enter the rich text editor (powered by Flutter Quill).
- Client establish real-time communication with backend via websocket.
- The client "joins" a specific Socket.IO "room" associated with that document's ID.
- **Real-time Synchronization:** As users type the Flutter Quill editor emits "delta" changes (a lightweight JSON format representing edits).
- These delta changes are sent via web socket to the backend, which then broadcasts them to all other clients in the same document's room, excluding the sender.
- **Auto-Save:** Periodically (every 2 seconds), the current document content (in delta format) is automatically sent to the backend via a Socket.IO event and saved to MongoDB, preventing data loss.



Working of App



4. Data Storage:

- User information and document content are stored in a MongoDB database.
- Document content is specifically stored in Quill's efficient delta format.



Working of App



5. Security & Session Management:

- All communication with the backend, especially for document management, occurs via RESTful APIs protected by JWT verification middleware.
- Routing within the Flutter app (using RouteMaster) ensures that document-related pages are only accessible to authenticated users.



Key Implementation



- **Google Sign-In:** Frontend `google_sign_in` package, Backend JWT generation/verification.
- **Real-time Collaboration:** Socket.IO for 'join-document' rooms, broadcasting Quill delta changes.
- **Auto-Save:** Client-side timer sends delta changes to backend via Socket.IO.
- **Rich Text Editor:** Flutter Quill for formatting, content stored in Quill's delta format in MongoDB.



Limitations



- Limited Offline Support.
- No Detailed Version History.
- Limited Media Support in editor.
- Scalability for Extreme Load (requires further optimization).



Challenged Encountered & Solutions



- Learning Curve with Flutter & Express
- Flutter State Management Complexity
- Learning websocket communication
- Google sign-in in our application where multiple exception is received



Beyond Final Defense(Future Scope)



- Comprehensive Offline Editing.
- Enhanced Media Embedding.
- Real-time Comments & Annotations.
- Desktop version of the app



Thank You!